

# DUSY: AI & API Integration & Architecture

## Notes

VIVA REVISION  
NOTES

Concept Explanations, Pipeline Flow, Code File Maps & Viva Defense Preparation

### 1. What Are the API and AI in This Project? (The Core Concepts)

#### What is the API? (The Bridge & Gateway)

Built using **FastAPI (Python)**. It is the structured interface between the user's React browser and the backend server. It handles security (JWT), data validation (Pydantic), saves data to MongoDB, and decides when and how to call the AI.

#### What is the AI? (The Clinical Brain)

Powered by **Google Gemini (Flash models)**. It is not just a chat widget; it is an **autonomous clinical agent** that interprets user symptoms, analyzes health baselines, checks for dangerous red flags, and prescribes recovery goals.

### 2. Why Are They Connected? (The Problem It Solves)

A standalone LLM (like opening ChatGPT or Gemini on a web browser) has **zero memory of the patient's medical records**. It does not know your height, weight, BMI, allergies, or sleep data unless you re-type it every single time.

#### The Core Engineering Innovation:

The **API connects the AI to the Patient Database**. Before the AI is ever called, the API silently fetches the patient's verified health profile (Height 185 cm, Weight 70 kg, BMI 20.5, Peanut Allergy, 7.5 hrs sleep) and injects it into the prompt. The AI answers with *exact awareness* of the user's body.

### 3. The 5-Step End-to-End Pipeline (How It Works from Click to Response)

- User Action (Frontend):** The user types a symptom (e.g. "*I have had a severe throbbing headache for 3 days*") or clicks an option chip in React.
- HTTP POST Request:** React dispatches a request to `POST /api/agent/interact` carrying the message and the user's secure **JWT Bearer Token**.
- API Interception & Context Assembly:**
  - FastAPI verifies the JWT token to identify the exact user.
  - The API queries MongoDB via Motor to assemble the patient's live vitals: BMI, chronic conditions, and allergies.
  - It loads past turns of the active consultation so the conversation maintains context.
- AI Agent Execution (Gemini Cascade):**
  - The API packages the patient vitals + clinical instructions + message history into a prompt.
  - It calls the **Google Gemini API** with strict JSON mode enabled.
  - Gemini reasons through the medical context and returns structured JSON (message text, clinical stage, clickable Choice Chips, and actionable recovery goals).
- Database Commit & Frontend Display:**
  - The API saves the conversation turn into the `agent_consultations` collection in MongoDB.
  - The JSON response is returned to React, which renders clean clinical typography, Choice Chips, and Goal Cards.

### 4. Where to See Their Connections in the Codebase (Exact File Map)

| File Path in Project                          | What It Does (The Connection Point)                                                                                                                                                                                                   |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app/routes/agent.py</code>              | <b>The API Entry Gate:</b> Defines the endpoints ( <code>/api/agent/interact</code> , <code>/api/agent/context</code> , <code>/api/agent/actions</code> ). It receives the HTTP request from React and forwards it to the AI service. |
| <code>app/services/agent_service.py</code>    | <b>The AI Brain &amp; Pipeline Engine:</b> The most crucial file (520+ lines). It initializes Gemini, builds the patient context block from MongoDB, enforces the 5-stage clinical intake state machine, and handles JSON parsing.    |
| <code>app/config.py</code>                    | <b>Credentials &amp; Configuration:</b> Securely reads <code>GEMINI_API_KEY</code> from the <code>.env</code> file so the backend can authenticate with Google's servers.                                                             |
| <code>app/models/agent.py</code>              | <b>Data Contracts (Pydantic v2):</b> Defines the exact JSON schemas exchanged between API, AI, and Frontend (e.g. <code>AgentResponse</code> , <code>AgentActionItem</code> ).                                                        |
| <code>frontend/src/pages/AgentPage.jsx</code> | <b>Frontend Consumer:</b> Dispatches <code>api.post('/api/agent/interact')</code> and renders the returned JSON into interactive Choice Chips and Goal Cards.                                                                         |

## 5. Deep-Dive: Key Technical Concepts to Know for Viva

### Concept 1: Dynamic Context Injection (Why It's Smarter Than a Chatbot)

In `agent_service.py`, before sending a query to Gemini, the backend queries MongoDB and injects a hidden telemetry block at the top of the prompt:

```
[ACTIVE PATIENT HEALTH TELEMETRY]
- Name: UJJWAL JHA | Age / Sex: Male
- Physical Baseline: Height 185 cm, Weight 70 kg, BMI: 20.5 (Normal)
- Documented Allergies: ['peanuts']
- Lifestyle: Sleep 7.5 hrs, Water 2.5 L, Daily Steps 8,500
```

**Viva Value:** If the user asks "What snack can I eat during recovery?", the AI will **never** suggest peanut butter because it has live awareness of the peanut allergy.

### Concept 2: Multi-Model Resilience Cascade (Zero-Downtime Architecture)

Cloud AI APIs can occasionally return 503 Service Unavailable errors due to traffic spikes. Instead of crashing, `agent_service.py` implements an automated cascading failover:

- **1st Attempt:** gemini-3.5-flash (Fastest, sub-second response).
- **2nd Attempt (Failover):** gemini-3.8-flash (Engaged automatically if 3.5 encounters a 503 error).
- **3rd Attempt (Failover):** gemini-3.5-flash-lite (Lightweight backup).
- **Deterministic Offline Engine:** `_smart_fallback_engine()` (Built-in rule engine that runs locally if the internet or API keys are down). The patient never sees a raw crash.

### Concept 3: Enforcing Structured JSON Mode (Why Markdown Backticks Break Code)

By default, LLMs return markdown text (e.g. ````json ... ````). If an API tries to parse that directly, it crashes with a `JSONDecodeError`. We solved this in two ways:

- We configure Gemini with `response_mime_type="application/json"`, forcing the model at the token level to output raw valid JSON.
- We added regex sanitizers in Python to strip backtick fences if any slip through before calling `json.loads()`.

### Concept 4: The 5-Stage Clinical Intake State Machine

Unlike casual chatbots that guess immediately, DUSY follows clinical protocols:

1. **Stage 1 (Intake):** Acknowledges symptom, probes for duration and frequency via Choice Chips.
2. **Stage 2 (Safety Screen):** Screens for red flags (sudden numbness, chest pain, high fever).
3. **Stage 3 (Guidance):** Synthesizes clinical advice correlated with patient BMI and hydration.
4. **Stage 4 (Goals):** Formulates 3 actionable, measurable recovery habits.
5. **Stage 5 (Completed):** Commits confirmed goals to MongoDB and syncs them to the dashboard.

## 6. What to Say to Your Teacher (Viva Presentation Speech)

*"Respected teachers, the core intelligence of DUSY comes from the integration between our **FastAPI backend** and **Google Gemini AI**.*

*The problem with generic chatbots is that they have no memory of the patient's medical baseline. In our system, the API acts as an intelligent intermediary:*

*When a patient speaks to DUSY, our FastAPI route first retrieves their verified health profile from MongoDB — including*

their BMI, chronic conditions, and allergies. The API dynamically injects this telemetry into Gemini's prompt context.

Furthermore, we do not let the AI output free-form unstructured paragraphs. We enforce **Strict JSON Mode**, requiring Gemini to return standardized payloads containing clinical stages, clickable Choice Chips, and actionable recovery goals. These recovery goals are then parsed by FastAPI and saved directly into MongoDB.

To guarantee enterprise reliability, we built a **Multi-Model Resilience Cascade**: if Gemini 3.5 experiences cloud traffic errors, our service automatically falls over to Gemini 3.8, Gemini Flash-Lite, and ultimately our deterministic local fallback engine. This ensures the application never crashes during clinical triage."

## 7. Quick-Fire Teacher Viva Q&A (Memorize These 4 Points)

### Q1: Why did you use Gemini Flash instead of GPT-4 or Gemini Pro?

Answer: "Gemini Flash offers sub-second inference latency (under 800ms) and low computational cost while maintaining high clinical reasoning accuracy, making it ideal for interactive conversational health triage."

### Q2: How do you prevent the AI from giving dangerous medical advice?

Answer: "Our AGENT\_SYSTEM\_PROMPT enforces clinical guardrails: it mandates red-flag triage screening before self-care guidance, explicitly forbids definitive diagnosing, and includes emergency escalation protocols."

### Q3: What happens if the Gemini API key expires or goes offline during your demo?

Answer: "Our `_smart_fallback_engine()` in `agent_service.py` detects the failure and immediately switches to deterministic offline state logic, providing valid Choice Chips and advice without throwing an unhandled exception."

### Q4: How does the frontend know which Choice Chips to display?

Answer: "The Gemini AI returns a `quick_replies` array inside its structured JSON response. React maps over this array and renders interactive clickable buttons dynamically."